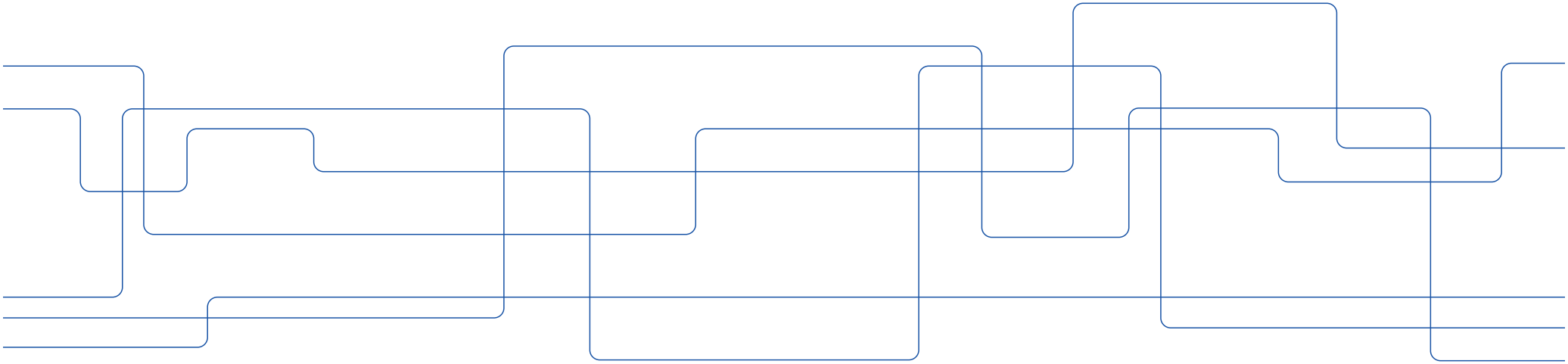


HI1024

Grundkurs i programmering

F3: Sekvens & Selektion!



Innehåll!

King 5

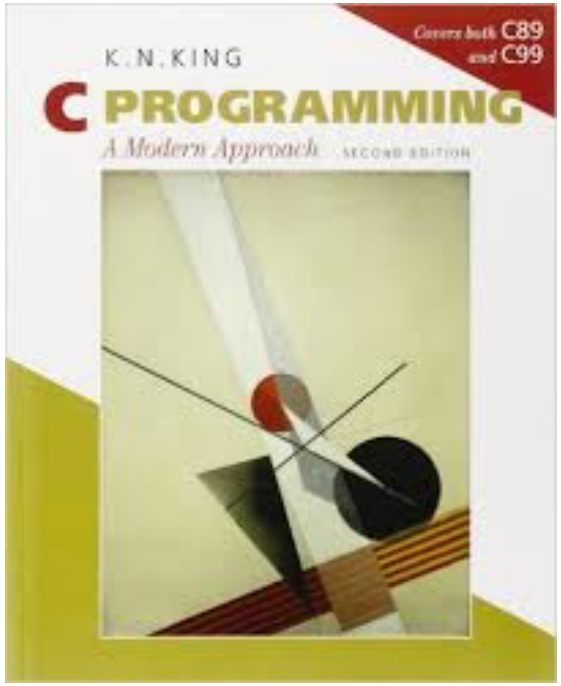
Repetition föreläsning #2

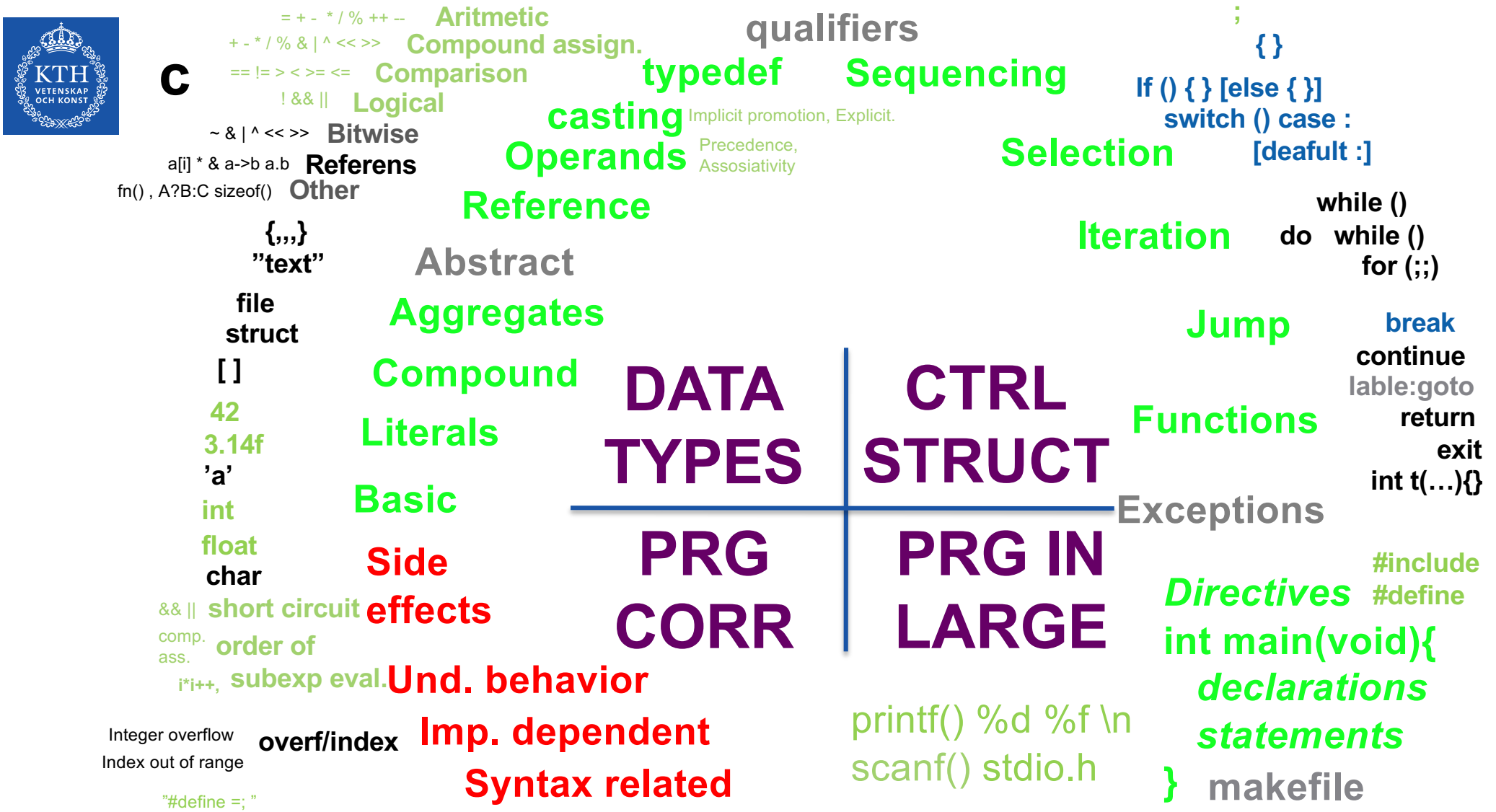
Sekvens (igen)

Selektion: if & switch

Sammansatt sats

Övning




C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
”text”
file
struct
[ ]
42
3.14f
’a’
int
float
char
Basic
Literals
Compound
Aggregates
Abstract
Reference
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence, Assosiativity
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
do while ()
for (;;)
Iteration
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
#include
#define
Directives
int main(void){
declarations
statements
}
makefile
&& || short circuit
comp. ass. order of
i*i++, subexp eval.
Side effects
Und. behavior
Imp. dependent
Syntax related
Integer overflow
Index out of range
overf/index
”#define =; ”
printf() %d %f \n
scanf() stdio.h
int/int
/*comment*/ or //

Nytt: scanf() i detalj…

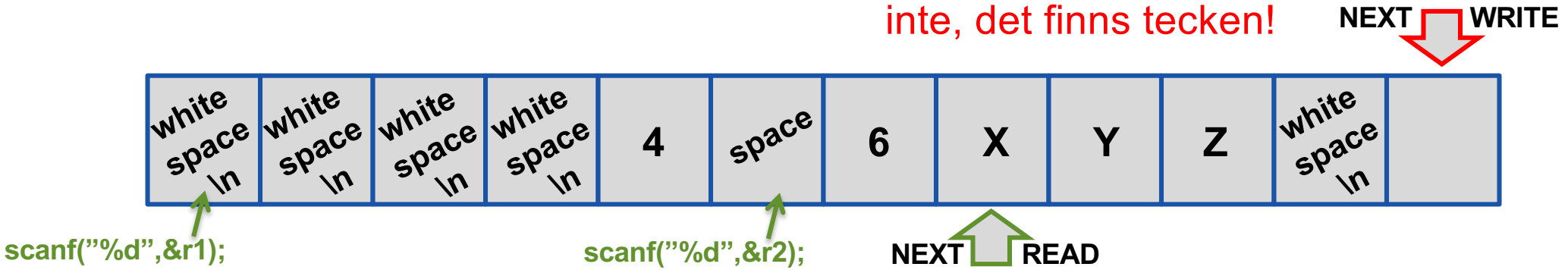



```
vcxxs-macbook:~ vcxx$ /Users/vcxx/Desktop/main
Parallel Resistor Calcylator (PRC)
First resistor  (ohm):
```

OS Keyboard buffer är tom, ”programmet” stannar!, du matar in [4*retur4 6YZretur]

```
4 6XYZ
Second resistor (ohm): Replace with a 2 ohm resistor.
PRC Done.
vcxxs-macbook:~ vcxx$
```

OBS: Programmet stannar inte, det finns tecken!



Väldigt vanligt fel, det ser ut som om den ”hoppar över” en inmatning!

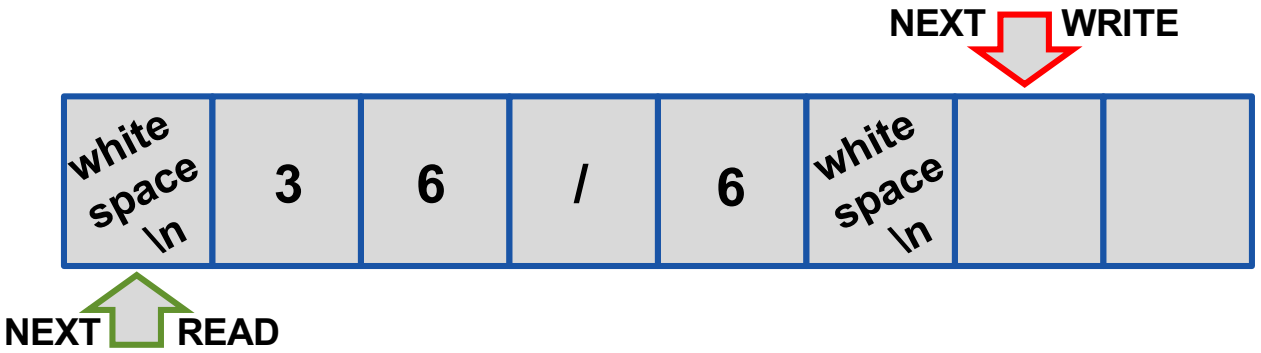
Nytt: scanf() i detalj…

R

Mönsterpassning!

Hitta en %d…
…som måste följas av ett ’/’…
…som måste följas av ett %d!

scanf(”%d/%d”,&numerator, &denominator);



Blanda inte ihop scanf:s mönsterpassning med printf:s styrtecken (aldrig \n i scanf)!

Gruppuppgift!

```c
#include <stdio.h>

int main(int argc, char **argv) {
    int year=62, month=9, day=5;
    printf("SSNb format converter\n");

    printf("Enter a SSNb (YY.MM.DD):");
    scanf("%d.%d.%d",&year, &month, &day);

    printf("Year  : %02d\n",year);
    printf("Month : %02d\n",month);
    printf("Day

    printf("SSNb f
    return 0;
}
```

```
SSNb format converter
Enter a SSNb (YY.MM.DD):62.05.09
Year  : 62
Month : 05
Day   : 09
SSNb format converter, bye.
```

1 Skriv ner problemet så att ett barn förstår det!

2 Gör ett minimalt main() av dina anteckningar!

3 Konstanter Variabler med Värde Uttryck & resultat!

a..n

4 Läs in riktiga värden!



Det finns några stycken…

= + - * / % ++ -- **Aritmetic**
+ - * / % & | ^ << >> **Compound assign.**
== != > < >= <= **Comparison**
! && || **Logical**
~ & | ^ << >> **Bitwise**
a[i] * & a->b a.b **Referens**
fn() , A?B:C sizeof() **Other**

…vi tittar på några av dem…
…men viktigast är nog att aldrig glömma bort att **använda parenteser ()!**

Precedence	Operator	Description	Associativity
1	++ -- () [] . –> (type){list}	Suffix/postfix increment and decrement Function call Array subscripting Structure and union member access Structure and union member access through pointer Compound literal(C99)	Left-to-right
2	++ -- + – ! ~ (type) * & sizeof _Alignof	Prefix increment and decrement Unary plus and minus Logical NOT and bitwise NOT Type cast Indirection (dereference) Address-of Size-of Alignment requirement(C11)	Right-to-left
3	* / %	Multiplication, division, and remainder	Left-to-right
4	+ –	Addition and subtraction	
5	<< >>	Bitwise left shift and right shift	
6	< <= > >=	For relational operators < and ≤ respectively For relational operators > and ≥ respectively	
7	== !=	For relational = and ≠ respectively	
8	&	Bitwise AND	
9	^	Bitwise XOR (exclusive or)	
10	|	Bitwise OR (inclusive or)	
11	&&	Logical AND	
12	||	Logical OR	
13[note 1]	?:	Ternary conditional[note 2]	Right-to-Left
14	= += –= *= /= %= <<= >>= &= ^= |=	Simple assignment Assignment by sum and difference Assignment by product, quotient, and remainder Assignment by bitwise left shift and right shift Assignment by bitwise AND, XOR, and OR	
15	,	Comma	Left-to-right

R

Aritmetiska

```
int r1=6, r2=4, rx, ry;

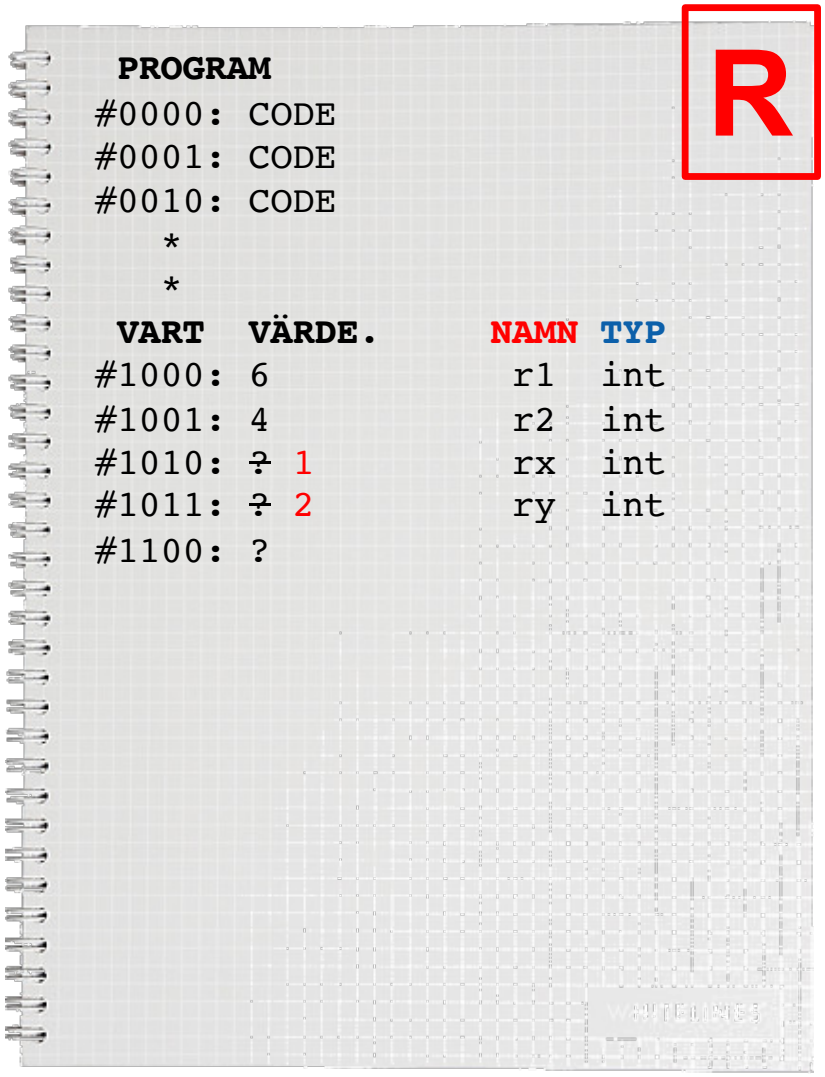
rx = r1 / r2;
ry = r1 % r2;

rx = rx + 5;

printf("%d\n", r1++);
printf("%d\n", ++r1);

rx+=5;
```

HL: % är rest vid heltalsdivision
HL: 6 % 4 är 2
VL: tilldelas 2!



% är otroligt användbart, t.ex x%2 avslöjar om x är ett udda eller jämt tal!

Aritmetiska

```
int r1=6, r2=4, rx, ry;

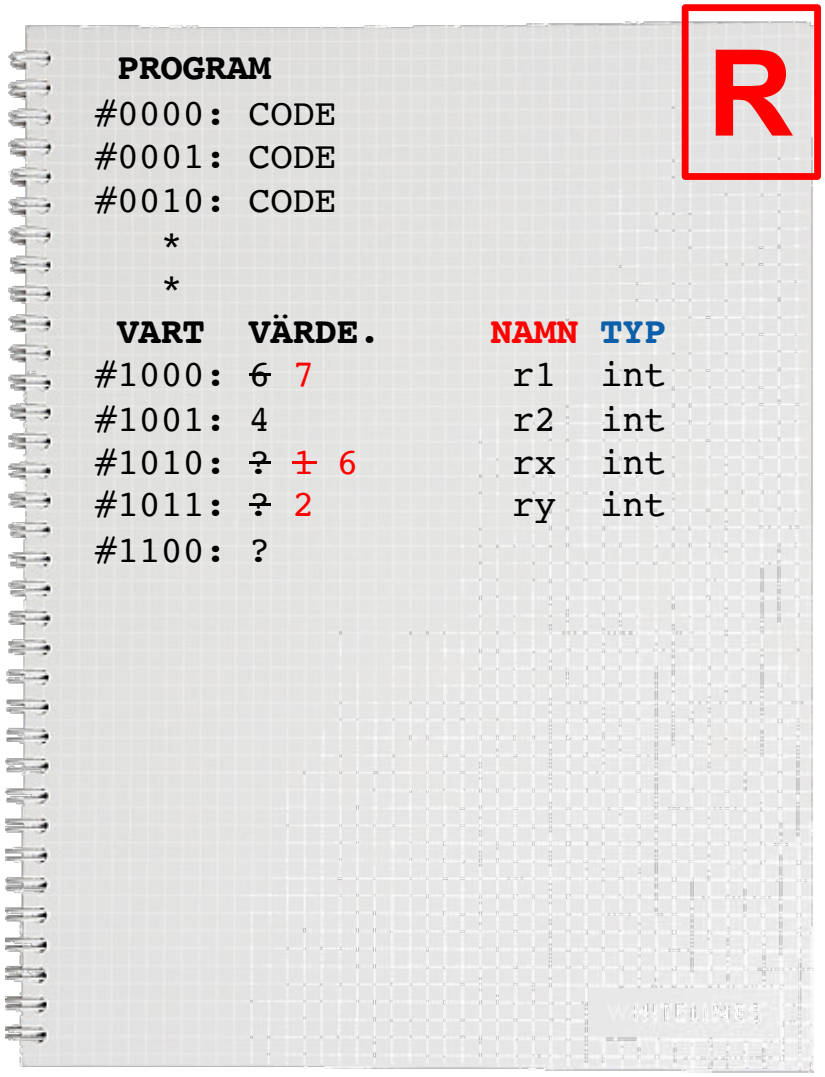
rx = r1 / r2;
ry = r1 % r2;

rx = rx + 5;

printf("%d\n", r1++);   (Skriver ut 6!)
printf("%d\n", ++r1);

rx+=5;
```

++ betyder öka värdet med 1
Står ++ **efter** variablen så används ursprungsvärdet i uttrycket!



Hämta r1 ursprungliga värde (6), skriv ut 6, öka r1 med 1 till 7 när utskriften är klar!

Aritmetiska

```
int r1=6, r2=4, rx, ry;

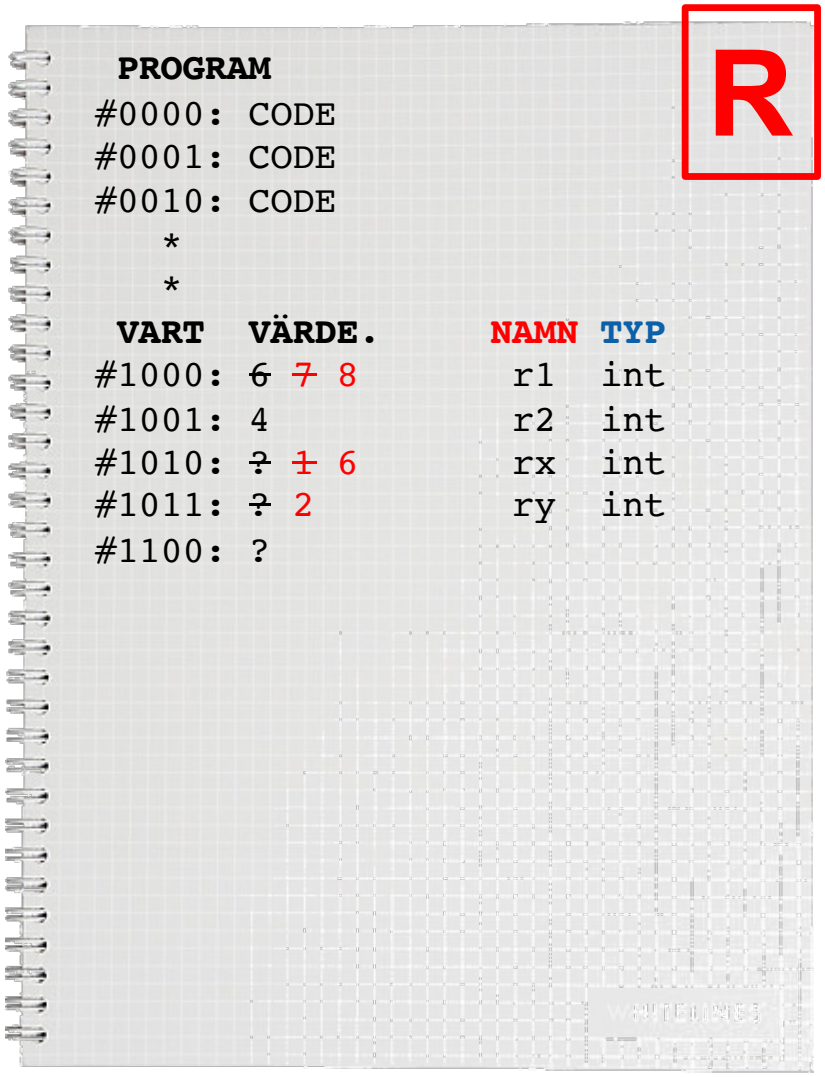
rx = r1 / r2;
ry = r1 % r2;

rx = rx + 5;

printf("%d\n", r1++);   (Skriver ut 6!)
printf("%d\n", ++r1);   (Skriver ut 8!)

rx+=5;
```

++ betyder öka värdet med 1
Står ++ **före** variablen så används det uppdaterade värdet i uttrycket!



Hämta r1 ursprungliga värde (7), öka r1 med 1 till 8, använd det nya värdet för utskriften!

Jämförande



Sant eller falsk representeras i c-språket på följande (lite märkliga) sätt:

Falskt : Heltalet 0

Sant. : Godtyckligt heltal utom 0, med fördel 1!

Operator	Meaning of Operator	Example
==	Equal to	5==3 returns false (0)
>	Greater than	5>3 returns true (1)
<	Less than	5<3 returns false (0)
!=	Not equal to	5!=3 returns true(1)
>=	Greater than or equal to	5>=3 returns true (1)
<=	Less than or equal to	5<=3 return false (0)

”Resultatet” av en jämförelse kan med andra ord skrivas ut med %d, eller användas som en int där det är tillåtet! x=x+(5>3); blir x=x+1;!

(Se upp med: = är tilldelning, == är lika med, det kan bli otroligt fel ibland!)



Logiska



Ofta behövs flera jämförande uttryck kombineras…
…det kan ske med de logiska operationerna:
AND, OR & NOT.

Operator	Meaning of Operator	Example
&&	Logial AND	If c=5 and d=2 then,((c==5) && (d>5)) returns false.
||	Logical OR	If c=5 and d=2 then, ((c==5) || (d>5)) returns true.
!	Logical NOT	If c=5 then, !(c==5) returns false.

AND

```
F F : F   Result must
F T : F   be F skip!
T F : F
T T : T
```

OR

```
F F : F
F T : T
T F : T   Result must
T T : T   be T skip!
```

NOT

```
F   : T
T   : F
```


(Se upp med && och || som bara utför den första testen om utfallet är givet, short-circuit!)

Övning #2: Reflektioner…

```c
#include <stdio.h>

int main(int argc, char **argv)
{
    int y=0, m=12, d=24;
    printf("US to EUR converter 1.0 AC\n");
    printf("Enter US date (MM/DD/YYY):");
    scanf("%d/%d/%d", &m, &d, &y);
    printf("EUR format:%d%02d%02d\n",y,m,d);
    printf("Program completed, bye!\n");
    return 0;
}
```

MM/DD/YYYY
To
YYYYMMDD

Mönstermatchning!

5/6*3/4
To
15/24

```c
scanf("%d/%d*%d/%d", &t1, &n1, &t2, &n2);
```

Bergis?!

- #include <stdio.h>
- #define MAX 38
- int m = 40;



- printf(”TEXT \n”);
- printf(”TEXT %d \n”, m);
- printf(”TEXT %d %d\n”, m, MAX);
- printf(”TEXT %d %d %d\n”, m, MAX, m>MAX);



Bergis?!

- #include <stdio.h>
- #define MAX 38
- int m = 40;



- printf(”TEXT \n”);
- printf(”TEXT %d \n”, m);
- printf(”TEXT %d %d\n”, m, MAX);
- printf(”TEXT %d %d %d\n”, m, MAX, m>MAX);

TEXT \n



Bergis?!

- #include <stdio.h>
- #define MAX 38
- int m = 40;



- printf(”TEXT \n”); TEXT \n
- printf(”TEXT %d \n”, m);
- printf(”TEXT %d %d\n”, m, MAX);
- printf(”TEXT %d %d %d\n”, m, MAX, m>MAX);

Bergis?!

- #include <stdio.h>
- #define MAX 38
- int m = 40;



- printf(”TEXT \n”); TEXT \n
- printf(”TEXT %d \n”, m); TEXT 40 \n
- printf(”TEXT %d %d\n”, m, MAX);
- printf(”TEXT %d %d %d\n”, m, MAX, m>MAX);



Bergis?!

- #include <stdio.h>
- #define MAX 38
- int m = 40;



- printf(”TEXT \n”); TEXT \n
- printf(”TEXT %d \n”, m); TEXT 40 \n
- printf(”TEXT %d %d\n”, m, MAX);
- printf(”TEXT %d %d %d\n”, m, MAX, m>MAX);

Bergis?!

- #include <stdio.h>
- #define MAX 38
- int m = 40;



• printf(”TEXT \n”);	TEXT \n
• printf(”TEXT %d \n”, m);	TEXT 40 \n
• printf(”TEXT %d %d\n”, m, MAX);	TEXT 40 38\n
• printf(”TEXT %d %d %d\n”, m, MAX, m>MAX);	



Bergis?!

- #include <stdio.h>
- #define MAX 38
- int m = 40;

- printf(”TEXT \n”); TEXT \n
- printf(”TEXT %d \n”, m); TEXT 40 \n
- printf(”TEXT %d %d\n”, m, MAX); TEXT 40 38\n
- printf(”TEXT %d %d %d\n”, m, MAX, m>MAX);



Bergis?!

- #include <stdio.h>
- #define MAX 38
- int m = 40;



• printf(”TEXT \n”);	TEXT \n
• printf(”TEXT %d \n”, m);	TEXT 40 \n
• printf(”TEXT %d %d\n”, m, MAX);	TEXT 40 38\n
• printf(”TEXT %d %d %d\n”, m, MAX, m>MAX);	TEXT 40 38 1\n



Bergis?!

- #include <stdio.h>
- int i,j,k;



- scanf(”%d”, i);
- scanf(”%d”, &i);
- scanf(”%d,%d”, &i, &j);
- scanf(”%d:%d:%d”, &i, &j, &k);
- scanf(”%d\n”, &i);

Bergis?!

scanf(”d%”, &i);
scanf(”&d”, &i);



- #include <stdio.h>
- int i,j,k;

- scanf(”%d”, i);
- scanf(”%d”, &i);
- scanf(”%d,%d”, &i, &j);
- scanf(”%d:%d:%d”, &i, &j, &k);
- scanf(”%d\n”, &i);

47\n

(om i=2 så försöker scanf() skriva 47 i minnescell 2 vilket helt säkert är illa!)

scanf() utgår från att det andra argumentet är vart i minnet variabel i finns, inte dess värde!

Bergis?!

scanf(”d%”, &i);
scanf(”&d”, &i);



- #include <stdio.h>
- int i,j,k;

- scanf(”%d”, i); 47\n
- scanf(”%d”, &i); 47\n i = 47
- scanf(”%d,%d”, &i, &j);
- scanf(”%d:%d:%d”, &i, &j, &k);
- scanf(”%d\n”, &i);

& framför en variabel avslöjar vart i minnet den finns lagrad, mycket användbart!

Bergis?!

scanf(”d%”, &i);
scanf(”&d”, &i);



- #include <stdio.h>
- int i,j,k;

- scanf(”%d”, i); 47\n
- scanf(”%d”, &i); 47\n i = 47
- scanf(”%d,%d”, &i, &j);
- scanf(”%d:%d:%d”, &i, &j, &k);
- scanf(”%d\n”, &i);

Bergis?!

scanf(”d%”, &i);
scanf(”&d”, &i);



- #include <stdio.h>
- int i,j,k;

- scanf(”%d”, i); 47\n
- scanf(”%d”, &i); 47\n — i = 47
- scanf(”%d,%d”, &i, &j); 5,7\n — i = 5 och j = 7
- scanf(”%d:%d:%d”, &i, &j, &k);
- scanf(”%d\n”, &i);

Bergis?!

scanf(”d%”, &i);
scanf(”&d”, &i);



- #include <stdio.h>
- int i,j,k;

• scanf(”%d”, i);	47\n	
• scanf(”%d”, &i);	47\n	i = 47
• scanf(”%d,%d”, &i, &j);	5,7\n	i = 5 och j = 7
• scanf(”%d:%d:%d”, &i, &j, &k);		
• scanf(”%d\n”, &i);		

Bergis?!

scanf(”d%”, &i);
scanf(”&d”, &i);



- #include <stdio.h>
- int i,j,k;

• scanf(”%d”, i);	47\n	
• scanf(”%d”, &i);	47\n	i = 47
• scanf(”%d,%d”, &i, &j);	5,7\n	i = 5 och j = 7
• scanf(”%d:%d:%d”, &i, &j, &k);	1:2:3\n	i = 1, j = 2 & k = 3
• scanf(”%d\n”, &i);		

Bergis?!

scanf(”d%”, &i);
scanf(”&d”, &i);



- #include <stdio.h>
- int i,j,k;

• scanf(”%d”, i);	47\n	
• scanf(”%d”, &i);	47\n	i = 47
• scanf(”%d,%d”, &i, &j);	5,7\n	i = 5 och j = 7
• scanf(”%d:%d:%d”, &i, &j, &k);	1:2:3\n	i = 1, j = 2 & k = 3
• scanf(”%d\n”, &i);		

Bergis?!

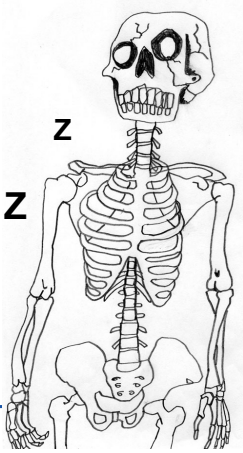
scanf(”d%”, &i);
scanf(”&d”, &i);



- #include <stdio.h>
- int i,j,k;

• scanf(”%d”, i);	47\n	
• scanf(”%d”, &i);	47\n	i = 47
• scanf(”%d,%d”, &i, &j);	5,7\n	i = 5 och j = 7
• scanf(”%d:%d:%d”, &i, &j, &k);	1:2:3\n	i = 1, j = 2 & k =
• scanf(”%d\n”, &i);	47\n	

Mönstermatchning



Hjälputskrifter…

- **Sträva alltid efter att ha ett fungerande program!**
- Lägg till 1-3 rader och provkör igen, fungerar det inte så är det mycket troligt att felet ligger i de senaste 1-3 raderna!
- **Använd ”extra” printf() för att i smyg visa vart du är i koden/variabler!**
- Ta inte bort de ”extra” printf():n när det fungera, gör om dem till kommentarer.

```
printf("Enter the diameter [cm]:");
scanf("%f", &diameter);
printf("DBG> diameter is [%f]\n",diameter);
printf("The circumference is %.2f[cm]\n",diameter*3.1415f);
```

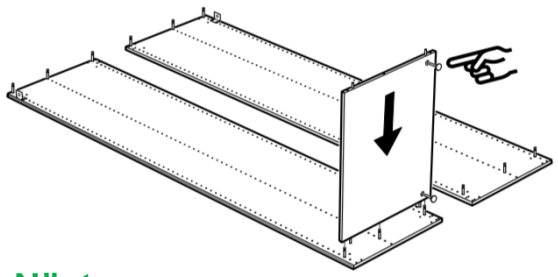
```
printf("Enter the diameter [cm]:");
scanf("%f", &diameter);
//printf("DBG> diameter is [%f]\n",diameter);
printf("The circumference is %.2f[cm]\n",diameter*3.1415f);
```

Sekvens (igen)…

1..

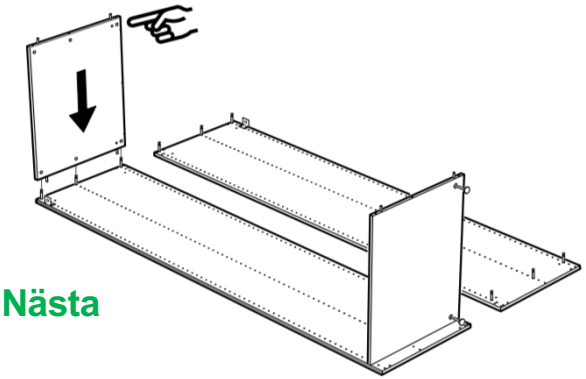
Nästa

10

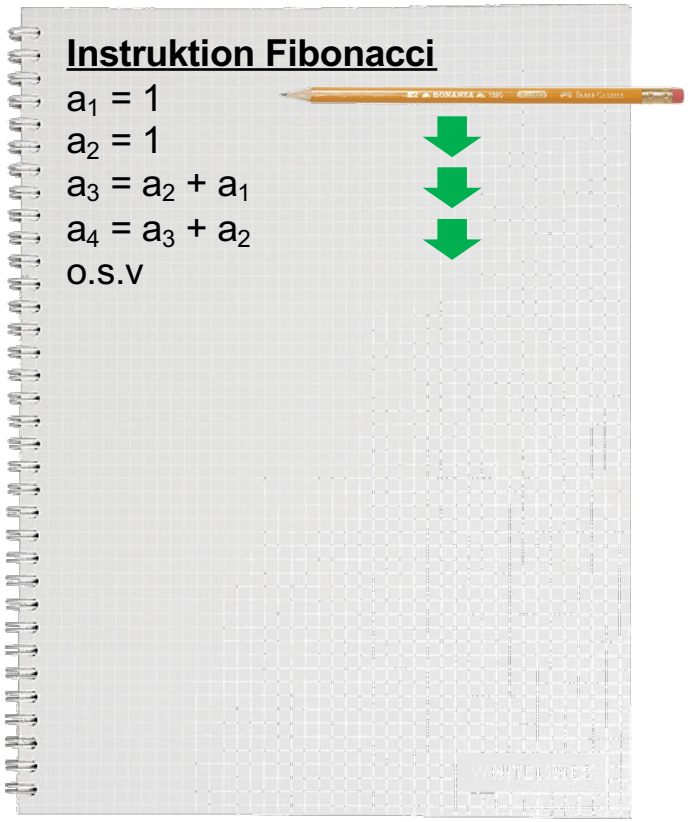


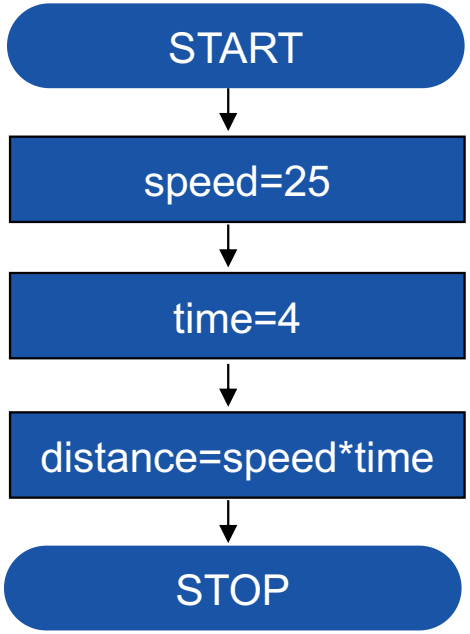
Nästa

11



Nästa



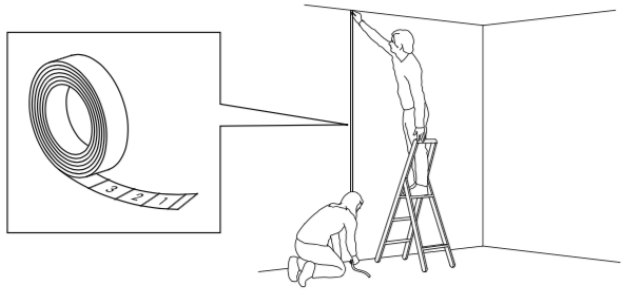


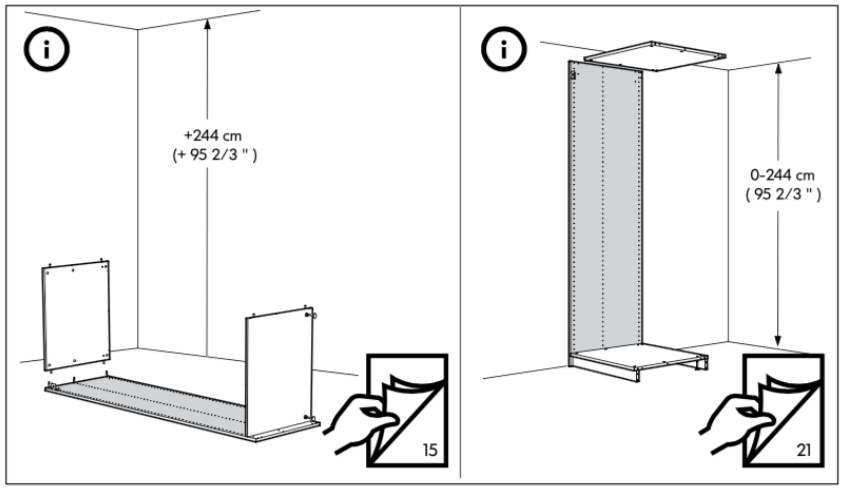
Flowchart

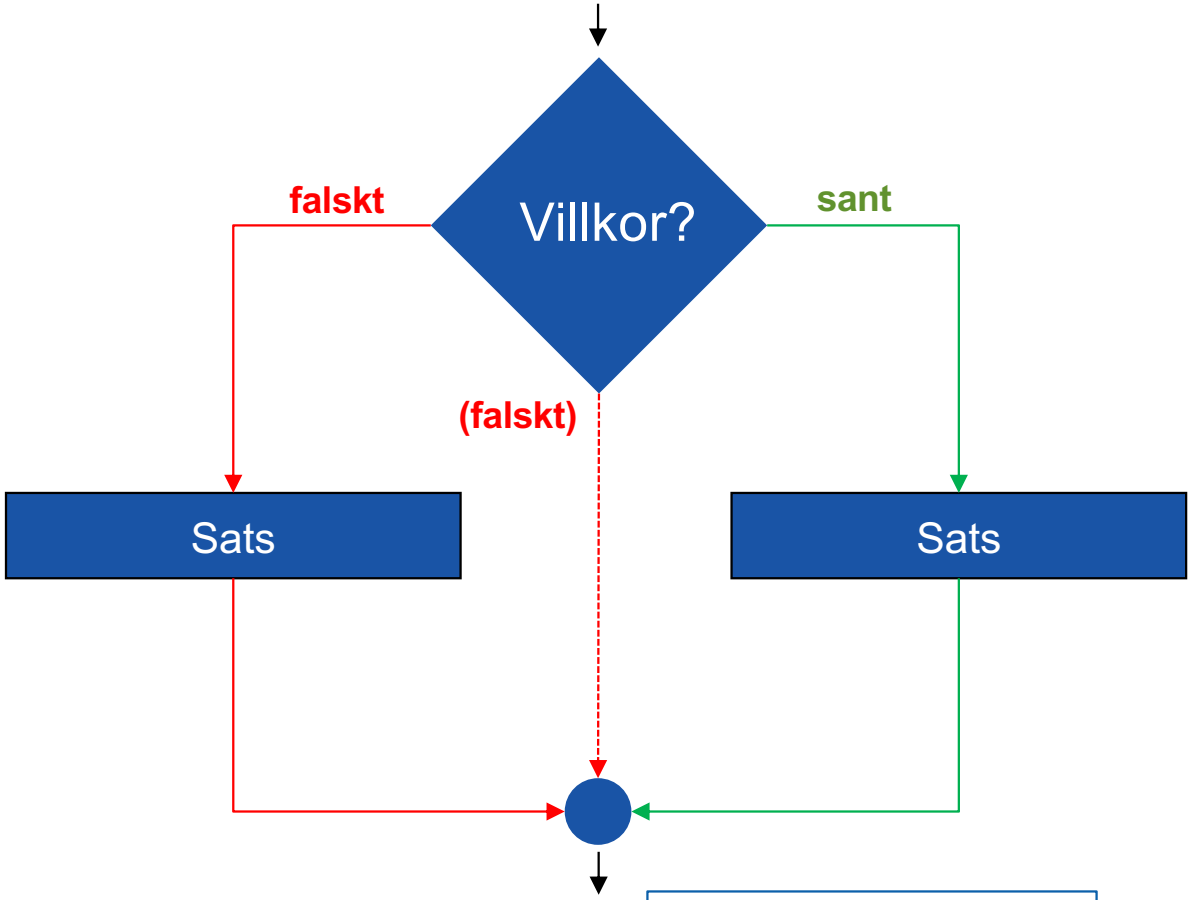
När du skriver t.ex. en tekniska rapporter kan du behöva förklara en algoritm. Sekvens beskrivs så här!

Selektion

9







Flowchart

Det behöver inte finnas någon falskt del, är villkoret falskt hoppar programmet bara över sant delen!

Selektion

Reserverat ord i c: **if**

Parenteser

Måste vara en (1) sats!

if (expression) statement

Ett uttryck som när det beräknas bli ett heltal, om det inte är 0 (sant) utförs statement!

```
printf("Mammal! : %d\n", legs<=4);
printf("Insect! : %d\n", ((legs>4) && (legs<8)));
printf("Spider! : %d\n", legs==8);
printf("Creepy! : %d\n",legs>8);
```



```
if (legs<=4) printf("Mammal!\n");
if ((legs>4) && (legs<8)) printf("Insect!\n");
if (legs==8) printf("Spider!\n");
if (legs>8) printf("Creepy!\n");
```


Om legs är 8 kommer enbart den 3:e testen att vara sann och programmet skriver ut ”Spider!”, och inget annat!

Sammansatt sats…

- Att det bara får stå en (1) sats efter villkoret är ett problem som måste lösas!

```
if (legs<=4) printf("Mammal!\n");
if ((legs>4) && (legs<8)) printf("Insect!\n");
if (legs==8) printf("Spider!\n");
if (legs>8) printf("Creepy!\n");
```

4 satser

```
{
    if (legs<=4) printf("Mammal!\n");
    if ((legs>4) && (legs<8)) printf("Insect!\n");
    if (legs==8) printf("Spider!\n");
    if (legs>8) printf("Creepy!\n");
}
```

1 sats sammansatt

OBS: Inget ´;´efter ´}´, viktigt! (Om det står där så är det en till (tom) sats, vilket kan bli jättefel!)

Selektion



Parenteser

Måste vara en (1) sats!

```
if ( expression ) statement#1 else statement#2
```

Ett uttryck som när det beräknas bli ett heltal, om det inte är 0 (S) utförs statement#1, om 0 (F) statement#2!

```
if (legs<=0) {
    printf("Ghost?!\n");
} else {
    if (legs<=4) printf("Mammal!\n");
    if ((legs>4) && (legs<8)) printf("Insect!\n");
    if (legs==8) printf("Spider!\n");
    if (legs>8) printf("Creepy!\n");
}
```

Det är helt ok, till och med smart, att göra en sammansatt sats av enbart en (1) sats – det är lätt att glömma bort { } om du/någon i framtiden vill lägga till en sats!

Övning

Pulsmonitor:

a) Om pulsen (beat) är utanför intervallet 40-80 spm,
Aktivera larmet (alarm 1), i annat fall stäng av larmet (0).

b) Skriv även ut texten larm på/av. c) och orsak!

```
// Konstant MAX 80
// Konstant MIN 40

// Heltal beat, alarm

// Om…
//
//
//
```

1

Övning

Pulsmonitor:

a) Om pulsen (beat) är utanför intervallet 40-80 spm,
Aktivera larmet (alarm 1), i annat fall stäng av larmet (0).

b) Skriv även ut texten larm på/av. c) och orsak!

```
// Konstant MAX 80
// Konstant MIN 40

// Heltal beat, alarm

// Om beat<MIN eller beat>MAX
//       Aktivera Alarm!
// i annat fall
//       Avaktivera alarm.
```

1

Övning

Pulsmonitor:

a) Om pulsen (beat) är utanför intervallet 40-80 spm,
Aktivera larmet (alarm 1), i annat fall stäng av larmet (0).

b) Skriv även ut texten larm på/av. c) och orsak!

```
// Konstant MAX 80
// Konstant MIN 40

// Heltal beat, alarm

// Om beat<MIN eller beat>MAX
//      Aktivera Alarm!
// i annat fall
//      Avaktivera alarm.
```

1 **(2)**

```
#define MAX 80
#define MIN 40

int beat, alarm;

if (beat<MIN || beat>MAX)
    alarm=1;
else
    alarm=0;
```

3

…också ok…

```
#define MAX 80
#define MIN 40

int beat, alarm;

if (beat>=MIN && beat<=MAX)
    alarm=0;
else
    alarm=1;
```

3

(4)

Var noga med gränser, speciellt på TEN2, dumt att tappa 1p på slarv!

Övning

Pulsmonitor:

a) Om pulsen (beat) är utanför intervallet 40-80 spm,
Aktivera larmet (alarm 1), i annat fall stäng av larmet (0).

b) Skriv även ut texten larm på/av. c) och orsak!

1

```
// Konstant MAX 80
// Konstant MIN 40

// Heltal beat, alarm

// Om beat>=MIN och beat<=MAX
//      Avaktivera alarm.
//      Visa: Normal puls 40-80spm
// i annat fall
//      Aktivera alarm!
//      Visa: Alarm!
```

Övning

Pulsmonitor:

a) Om pulsen (beat) är utanför intervallet 40-80 spm,
Aktivera larmet (alarm 1), i annat fall stäng av larmet (0).

b) Skriv även ut texten larm på/av. c) och orsak!

```
// Konstant MAX 80
// Konstant MIN 40

// Heltal beat, alarm

// Om beat>=MIN och beat<=MAX
//      Avaktivera alarm.
//      Visa: Normal puls 40-80spm
// i annat fall
//      Aktivera alarm!
//      Visa: Alarm!
```

1 (2)

```
#define MAX 80
#define MIN 40

int beat, alarm;

if (beat>=MIN && beat<=MAX) {
    alarm=0;
    printf("Pulsen 40-80spm!");
} else {
    alarm=1;
    printf("!Alarm! !Alarm!");
}
```

3 (4)



Övning

Pulsmonitor:

a) Om pulsen (beat) är utanför intervallet 40-80 spm,
Aktivera larmet (alarm 1), i annat fall stäng av larmet (0).

b) Skriv även ut texten larm på/av. c) och orsak!

```
// Konstant MAX 80
// Konstant MIN 40

// Heltal beat, alarm

// Om beat>=MIN och beat<=MAX
//      Avaktivera alarm.
//      Visa: Normal puls 40-80spm
// i annat fall
//      Aktivera alarm!
//      Om beat<40
//          Visa: Alarm: Puls under 40!
//      i annat fall
//          Visa: Alarm: Puls över 80!
```

1

Hum, en if i en if, en nästlad if-sats?!

Övning

Pulsmonitor:

a) Om pulsen (beat) är utanför intervallet 40-80 spm,
Aktivera larmet (alarm 1), i annat fall stäng av larmet (0).

b) Skriv även ut texten larm på/av. c) och orsak!

```
// Konstant MAX 80
// Konstant MIN 40

// Heltal beat, alarm

// Om beat>=MIN och beat<=MAX
//      Avaktivera alarm.
//      Visa: Normal puls 40-80spm
// i annat fall
//      Aktivera alarm!
//      Om beat<40
//          Visa: Alarm: Puls under 40!
//      i annat fall
//          Visa: Alarm: Puls över 80!
```

1 (2)



3 (4)

```
#define MAX 80
#define MIN 40

int beat, alarm;

if (beat>=MIN && beat<=MAX) {
    alarm=0;
    printf("Pulsen 40-80spm!");
} else {
    alarm=1;
    if (beat<MIN)
        printf("!Alarm! Pulsen under 40!!!");
    else
        printf("!Alarm! Pulsen över 80!!!");
}
```

Bergis?!

Blodtrycksvakt:

Om det systoliska blodtrycket överstiger 130, eller det diastoliska blodtrycket överstiger 90 ska det skrivas ut på skärmen, i annat fall ska texten blodtrycket ok skrivas ut.

```
// Konstant SYSMAX 130
// Konstant DIAMAX 90

// Heltal sys, dia

// Om sys<=SYSMAX och dia<=DIAMAX
//       Visa: Blodtrycket är ok!
// i annat fall
//       Om sys>SYSMAX
//            Visa: Alarm: Systoliskt!!!
//       i annat fall
//            Visa: Alarm: Diastoliskt!!!
```

1

Bergis?!

Blodtrycksvakt:

Om det systoliska blodtrycket överstiger 130, eller det diastoliska blodtrycket överstiger 90 ska det skrivas ut på skärmen, i annat fall ska texten blodtrycket ok skrivas ut.

```
// Konstant SYSMAX 130
// Konstant DIAMAX 90

// Heltal sys, dia

// Om sys<=SYSMAX och dia<=DIAMAX
//      Visa: Blodtrycket är ok!
// i annat fall
//      Om sys>SYSMAX
//          Visa: Alarm: Systoliskt!!!
//      i annat fall
//          Visa: Alarm: Diastoliskt!!!
```

1 (2)

```
#define SYSMAX 130
#define DIAMAX 90

int systoliska, diatoliska;

if (systoliska<=SYSMAX && diatoliska<=DIAMAX) {
    printf("Blodtrycket är ok...");
} else {
    if (systoliska>SYSMAX)
        printf("!Alarm! Systoliskt!!!");
    else
        printf("!Alarm! Diastoliskt!!!");
}
```

3 (4)

Ett annat sätt att göra selektion…

- If-satsen är otroligt bra för att göra två olika saker beroende på om ett uttryck är sant eller falskt, men tänk om koden ska göra 7 olika saker beroende på värdet av en styrande variabel – det blir väldigt många (nästlade) if-sater….

```
if (theWeekDay==0) {
    printf("Monday");
} else {
    if (theWeekDay==1) {
        printf("Tuesday");
    } else {
        if (theWeekDay==2) {
            printf("Wednesday");
        } else {
            /* 4 more times... */
        }
    }
}
```

```
if (theWeekDay==0) printf("Monday");
if (theWeekDay==1) printf("Tuesday");
if (theWeekDay==2) printf("Wednesday");
if (theWeekDay==3) printf("Thursday");
if (theWeekDay==4) printf("Friday");
if (theWeekDay==5) printf("Saturday");
if (theWeekDay==6) printf("Sunday");
```

Den högra varianten är lite mer kompakt då det enbart är en sats som ska utföras, men det blir många tester i onödan oavsett vilken dag det är!

Switch-satsen…

Nya reserverade ord:
switch, case & default

```
switch ( expression ) {
  case constant-expression : statements
                …
  case constant-expression : statements
  default : statements /* kan utelämnas! */
}
```

expression **måste** bli ett **heltal**!
constant-expression **måste** vara en **konstant**!
statements behöver **inte** inneslutas i { } **men**
exekveringen fortsätter, även in i nästa case,
till det finns en **break-sats**;

```
if (theWeekDay==0) {
    printf("Monday");
} else {
    if (theWeekDay==1) {
        printf("Tuesday");
    } else {
        if (theWeekDay==2) {
            printf("Wednesday");
        } else {
            /* 4 more times... */
        }
    }
}
```



```
switch (theWeekDay) {
    case  0 : printf("Monday"); break;
    case  1 : printf("Tuesday"); break;
    case  2 : printf("Wednesday"); break;
    case  3 : printf("Thursday"); break;
    case  4 : printf("Friday"); break;
    case  5 : printf("Saturday"); break;
    case  6 : printf("Sunday"); break;
    default : printf("Not a weekday!");
}
```

Otroligt mycket bättre ur läsbarhetssynvinkel, leder till snabbare kod & möjlighet att notera fel!

Switch-satsen, fördjupning…

```c
int main(int argc, char **argv) {
    int score = 8;      //0-5 Fail, 6-7 Pass & 8-9 with dist!

    printf("Test Result 1.0\n");

    switch (score) {
        case 6 :
        case 7 : printf("Pass!\n"); break;
        case 8 :
        case 9 : printf("Pass with distinction!\n"); break;
        default: printf("Failed!");
    }

    printf("Exit TR!");
    return 0;
}
```

```
Test Result 1.0
Pass with distinction!
```


Tyvärr kan constant-expression inte vara en range, men är det få val kan utelämnade break fungera bra!

Switch-satsen, fördjupning…

```c
#include <stdio.h>
#include <stdbool.h>
#define MAX 50

int main(int argc, char **argv) {
    int speed = 60;

    printf("Speed Control 1.0\n");

    switch (speed>MAX) {
        case true : printf("*** To fast, slow down!\n"); break;
        case false: printf("*** Nice and steady, continue!\n"); break;
    }

    printf("Exit SC!");
    return 0;
}
```

Nytt bibliotek som definierar bl.a. true till 1 och false till 0!

Det går utmärkt, ibland t.o.m smart (läsbarhet), att använda en switch i stället för en if/else!



Bergis?!

Du vill skriva ut spade, heart, diamond, club beroende på värdet på variabeln theSuit (0..3), lös uppgiften med switch!

```
switch ( expression ) {
  case constant-expression : statements
                  …
  case constant-expression : statements
  default : statements /* kan utelämnas! */
}
```

expression **måste** bli ett **heltal**!
constant-expression **måste** vara en **konstant**!
statements behöver **inte** inneslutas i { } **men** exekveringen fortsätter, även in i nästa case, till det finns en **break-sats**;

```
// Om theSuit
//    är spader : visa Spader, klart
//    är hjärter : visa Hjärter, klart
//    är ruter    : visa Ryter, klart
//    är klöver  : visa klöver, klart
//    i annat fall: visa programfel!
```

Bergis?!

Du vill skriva ut spade, heart, diamond, club beroende på värdet på variabeln theSuit (0..3), lös uppgiften med switch!

```
switch ( expression ) {
  case constant-expression : statements
                  …
  case constant-expression : statements
  default : statements /* kan utelämnas! */
}
```

expression **måste** bli ett **heltal**!
constant-expression **måste** vara en **konstant**!
statements behöver **inte** inneslutas i { } **men** exekveringen fortsätter, även in i nästa case, till det finns en **break-sats**;

```
// Om theSuit
//    är spader : visa Spader, klart
//    är hjärter : visa Hjärter, klart
//    är ruter    : visa Ryter, klart
//    är klöver  : visa klöver, klart
//    i annat fall: visa programfel!
```

1 **3**

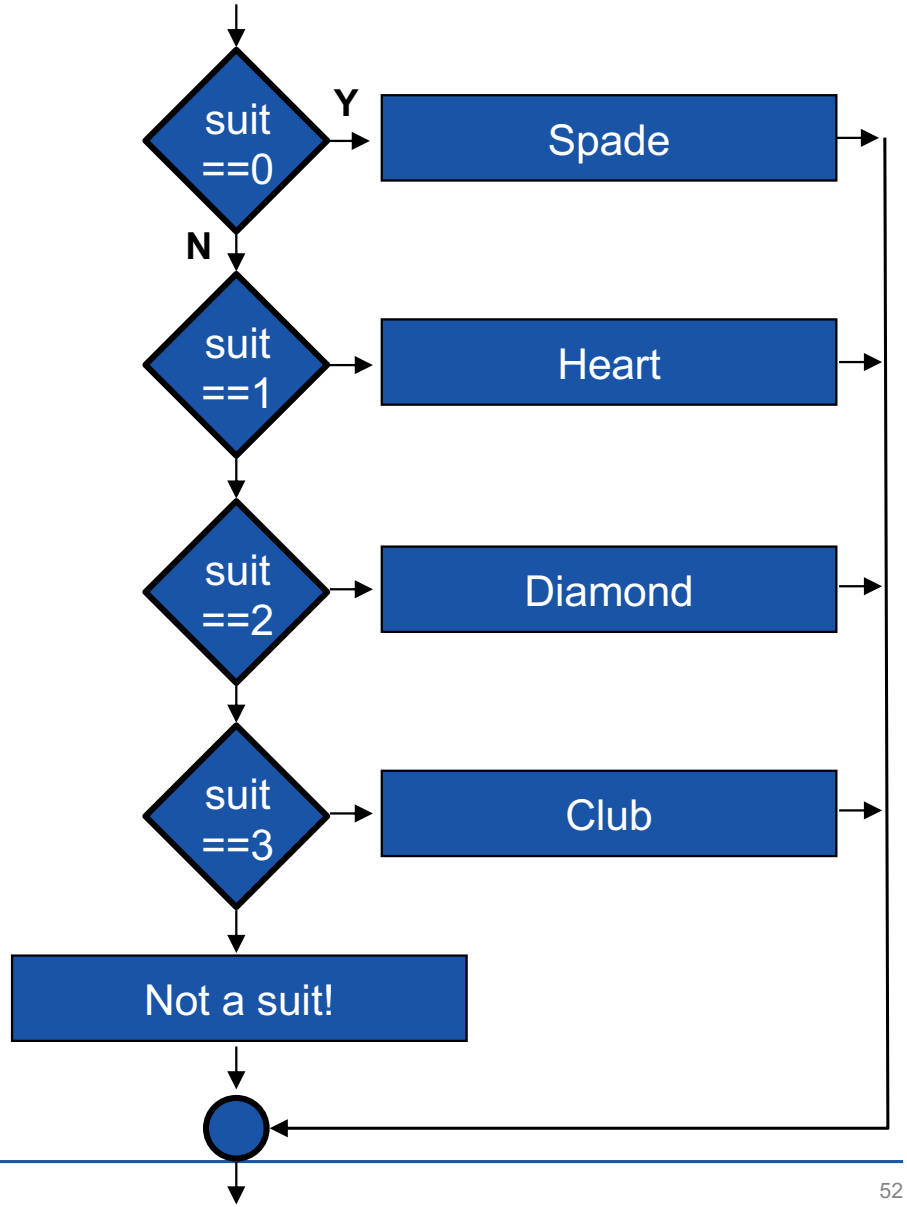
(2) **(4)**

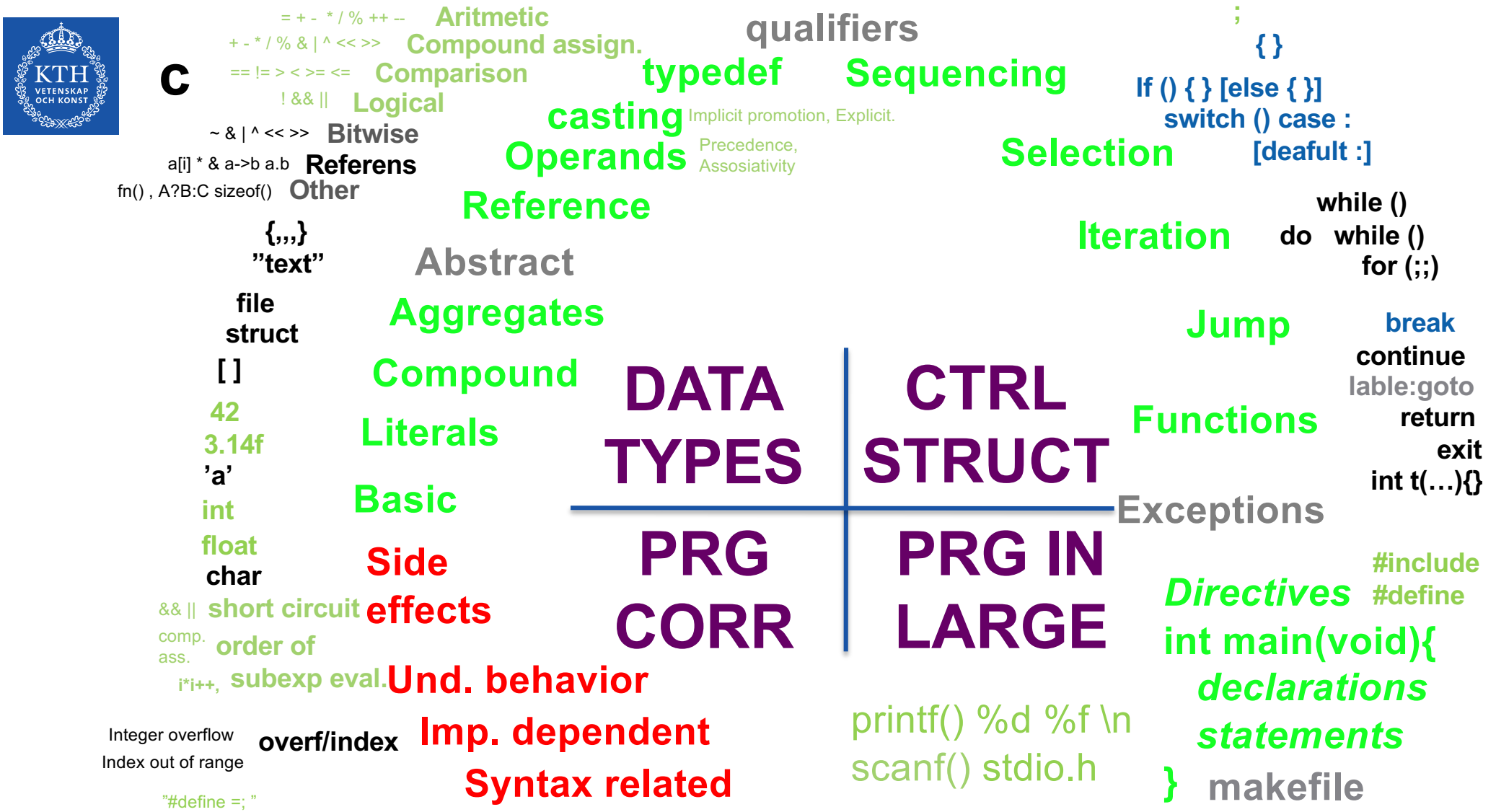
```
switch (theSuit) {
    case  0 : printf("Spade    "); break;
    case  1 : printf("Heart    "); break;
    case  2 : printf("Diamond  "); break;
    case  3 : printf("Club     "); break;
    default : printf("Not a suit!");
}
```

Flowchart

```
// Om theSuit
//    är spader : visa Spader, klart
//    är hjärter : visa Hjärter, klart
//    är ruter    : visa Ryter, klart
//    är klöver  : visa klöver, klart
//    i annat fall: visa programfel!
```

```
switch (theSuit) {
    case  0 : printf("Spade    "); break;
    case  1 : printf("Heart    "); break;
    case  2 : printf("Diamond  "); break;
    case  3 : printf("Club     "); break;
    default : printf("Not a suit!");
}
```




C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
”text”
file
struct
[ ]
42
3.14f
’a’
int
float
char
Basic
Literals
Compound
Aggregates
Abstract
Reference
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence, Assosiativity
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
do while ()
for (;;)
Iteration
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
#include
#define
Directives
int main(void){
declarations
statements
}
makefile
&& || short circuit
comp. ass.
order of
i*i++,
subexp eval.
Side effects
Und. behavior
Imp. dependent
Syntax related
Integer overflow
Index out of range
overf/index
”#define =; ”
printf() %d %f \n
scanf() stdio.h
int/int
/*comment*/ or //



Övningar #3: Kapitel 5:P1,P7 & E11

1 **Skriv ner proble-met så att ett barn förstår det!**

374
has
3 digits

21 43 10 35
Max & min
43, 10

229	Albany
404	Atlanta
470	Atlanta
478	Macon
678	Atlanta
706	Columbus
762	Columbus
770	Atlanta
912	Savannah
Other	not recognized

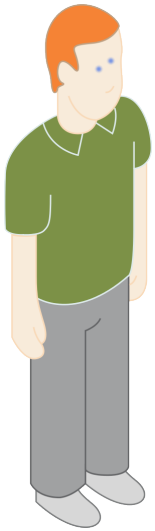
2 **Gör ett minimalt main() av dina anteckningar!**

3 **Konstanter Variabler med Värde Uttryck & resultat!**
a..n

4 **Läs in riktiga värden!**

Bonus: Gör om ”ben-analysatorn” med switch!

Historik



- 150701 0.1 AC Skapad.
- 150702 0.2 AC Uttryck & operationer.
- 150703 0.5 AC Övningsexempel.
- 150704 0.9 AC Allt utom case.
- 150706 1.0 AC Rev A
- 160808 1.1 AC Mindre uppdateringar och test uppgift sist.
- 170802 1.2 AC Förtydliganden av prg. Metodik samt utökning av switch-avsnittet.
- 190801 2.0 AC Omarbetad från grunden, samt anpassad till KTH:s nya profil.